

Docker 容器化部署从入门到精通

Docker 容器化部署从入门到精通

文档类型：阶梯式教程 适用读者：后端开发 / DevOps 工程师 前置知识：Linux 命令行基础
学习周期：约 2 周完成全部实验**文档类型**：阶梯式教程
适用读者：后端开发 / DevOps 工程师
前置知识：Linux 命令行基础
学习周期：约 2 周完成全部实验
文档类型：阶梯式教程 适用读者：后端开发 / DevOps 工程师 前置知识：Linux 命令行基础
学习周期：约 2 周完成全部实验**文档类型**：阶梯式教程
适用读者：后端开发 / DevOps 工程师
前置知识：Linux 命令行基础
学习周期：约 2 周完成全部实验

1. Docker 基础概念1. Docker 基础概念

1.1 什么是容器？1.1 什么是容器？

容器是一种操作系统级虚拟化技术，将应用及其依赖打包到一个轻量级、可移植的单元中。与虚拟机不同，容器共享宿主机内核，启动速度达到毫秒级。

1.2 核心组件对比1.2 核心组件对比

概念概念	说明说明	类比类比
Image (镜像) **Image (镜像) **	只读模板，包含运行应用所需的一切只读模板	Class 类定义Class 类定义
Container (容器) **Container (容器) *	镜像的运行实例镜像的运行实例	Object 对象实例Object 对象实例
Dockerfile**Dockerfile**	构建镜像的指令文件构建镜像的指令文件	Makefile / 配方Makefile / 配方
Registry**Registry**	存储和分发镜像的仓库存储和分发镜像的仓库	Maven 仓库 / npm registryMaven 仓库 / n
Volume**Volume**	持久化数据存储持久化数据存储	外挂硬盘外挂硬盘
Network**Network**	容器间通信网络容器间通信网络	虚拟交换机虚拟交换机

2. 安装与环境配置2. 安装与环境配置

2.1 Ubuntu/Debian 安装2.1 Ubuntu/Debian 安装

```
#
sudo apt-get remove docker docker-engine docker.io containerd runc

#
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg lsb-release

# Docker GPG
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

#
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin

#
sudo docker run hello-world

# docker sudo
sudo usermod -aG docker $USER
newgrp docker
```

3. Dockerfile 编写3. Dockerfile 编写

3.1 多阶段构建3.1 多阶段构建

```
# =====
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force

COPY . .
RUN npm run build

# =====
FROM node:20-alpine AS runner

RUN addgroup -g 1001 -S appgroup && \
    adduser -S appuser -u 1001 -G appgroup

WORKDIR /app
COPY --from=builder --chown=appuser:appgroup /app/dist ./dist
COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules
COPY --chown=appuser:appgroup package.json ./

EXPOSE 3000
USER appuser
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD wget -qO- http://localhost:3000/health || exit 1

ENTRYPOINT ["node", "dist/main.js"]
```

3.2 Dockerfile 最佳实践3.2 Dockerfile 最佳实践

原则原则	说明说明	错误示例错误示例	正确示例正确示例
最小化层数最小化层数	合并 RUN 指令合并 RUN 指令	多个独立 RUN多个独立 RUN	RUN cmd1 && cmd2`RUN cmd1 && cmd2`
利用缓存利用缓存	先拷贝依赖文件先拷贝依赖文件	COPY . . 在前`COPY . .` 在前	COPY package.json 在前`COPY package.j
非 root 运行非 root 运行	安全性安全性	默认 root默认 root	USER appuser`USER appuser`
.dockerignore.dockerignore	减小构建上下文减小构建上下文	无 .dockerignore无 .dockerigi	排除 node_modules排除 node_modules
精确标签精确标签	避免 latest避免 latest	FROM node:latest`FROM noc	FROM node:20.11-alpine`FROM node:20
健康检查健康检查	运行时健康监控运行时健康监控	无 HEALTHCHECK无 HEALTHC	添加 HEALTHCHECK添加 HEALTHCHECK

4. Docker Compose 编排4. Docker Compose 编排

4.1 多服务应用编排4.1 多服务应用编排

```
version: "3.9"
services:
  postgres:
    image: postgres:16-alpine
    container_name: app-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: app_production
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    networks:
      - backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: app-cache
    command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy allkeys-lru
    volumes:
      - redisdata:/data
    networks:
      - backend

  api:
    build:
      context: .
      dockerfile: Dockerfile
      args:
        NODE_ENV: production
    container_name: app-api
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/app_production
      REDIS_URL: redis://redis:6379
      NODE_ENV: production
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    networks:
      - backend
      - frontend

  nginx:
    image: nginx:1.25-alpine
    container_name: app-proxy
    ports:
      - "80:80"
      - "443:443"
    volumes:
```


5. 网络模型5. 网络模型

5.1 网络驱动对比5.1 网络驱动对比

驱动驱动	隔离性隔离性	多主机多主机	性能性能	使用场景使用场景
bridgebridge	中（同一宿主机）中（同		高高	单机多容器单机多容器
hosthost	无（共享宿主机）无（共		最高最高	高性能需求高性能需求
overlayoverlay	高高		中中	Swarm 集群Swarm 集群
macvlanmacvlan	中中		高高	直接暴露到物理网络直接暴露到物理网络
nonenone	完全隔离完全隔离	--	--	安全敏感场景安全敏感场景

5.2 自定义网络配置5.2 自定义网络配置

```
#
docker network create \
  --driver bridge \
  --subnet 172.20.0.0/16 \
  --gateway 172.20.0.1 \
  --opt com.docker.network.bridge.name=app-bridge \
  app-network

#
docker network inspect app-network

#
docker network connect app-network my-container
```

6. 数据管理6. 数据管理

6.1 Volume 类型6.1 Volume 类型

类型类型	存储位置存储位置	持久性持久性	适用场景适用场景
VolumeVolume	/var/lib/docker/volumes/`/va	容器删除后保留 容器删除后保	数据库、应用数据数据库、应用数据
Bind MountBind Mount	宿主机任意路径宿主机任意路径		开发环境热更新开发环境热更新
tmpfstmpfs	内存内存	重启清空 重启清空	临时敏感数据临时敏感数据

7. CI/CD 集成7. CI/CD 集成

7.1 GitHub Actions 示例7.1 GitHub Actions 示例

```
name: Build and Push Docker Image

on:
  push:
    tags:
      - 'v*'

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and Push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${ github.repository }:latest
            ghcr.io/${ github.repository }:${ github.ref_name }
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

8. 常见问题排查8. 常见问题排查

问题问题	排查命令排查命令	解决方案解决方案
端口冲突端口冲突	<code>docker ps -a`docker ps -a`</code>	更换端口映射更换端口映射
磁盘空间不足磁盘空间不足	<code>docker system df`docker system df`</code>	<code>docker system prune -a`docker syste</code>
容器无法启动容器无法启动	<code>docker logs <container>`docker logs</code>	查看日志定位错误查看日志定位错误
镜像拉取慢镜像拉取慢	——	配置国内镜像加速器配置国内镜像加速器
内存不足内存不足	<code>docker stats`docker stats`</code>	限制容器内存 <code>--memory</code> 限制容器内存 <code>`--memory`</code>

9. 总结9. 总结

本教程从 Docker 基础概念入手，逐步深入到多阶段构建、Compose 编排、网络配置和生产环境部署。以

下为学习路线建议：

1. 第 1 周：安装 Docker，运行第一个容器，理解镜像和容器的区别**第 1 周**：安装 Docker，运行第一个容器，理解镜像和容器的区别
2. 第 2 周：编写 Dockerfile，实践多阶段构建和优化技巧**第 2 周**：编写 Dockerfile，实践多阶段构建和优化技巧
3. 第 3 周：使用 Docker Compose 编排多服务应用**第 3 周**：使用 Docker Compose 编排多服务应用
4. 第 4 周：深入网络和存储，接入 CI/CD 流水线**第 4 周**：深入网络和存储，接入 CI/CD 流水线

技巧：每天花 15 分钟阅读 Docker 官方文档，结合动手实践效果最佳。 **技巧**：每天花 15 分钟阅读 [Docker 官方文档](https://docs.docker.com/)，结合动手实践效果最佳。

技巧：每天花 15 分钟阅读 Docker 官方文档，结合动手实践效果最佳。 **技巧**：每天花 15 分钟阅读 [Docker 官方文档](https://docs.docker.com/)，结合动手实践效果最佳。

本文档采用 CC BY-SA 4.0 许可协议。